Embedding Models, Linear Probing, CLIP Zero-Shot & Retrieval

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 5, Day 33, Self-study

21 October 2026

Table of Contents

1. Where This Fits
2. Learning Outcomes
3. Embedding Models: Where the Vector Comes From
4. Linear Probing as a Measurement
5. CLIP Zero-Shot in Depth
6. Retrieval in Depth
7. Olympiad Practice
8. Check Yourself
9. Limitations and What Comes Next
10. Summary
11. References

From the Lecture

- **In the lectures:** Day 30 named the linear probe (a `Linear` head on frozen features). Day 31 made the frozen vector the object: an **embedding**, cosine similarity, `E = F.normalize(...)`, `S = E @ E.T`; search, kNN and the probe on *cached* embeddings; CLIP's two encoders and contrastive loss; zero-shot from prompts; retrieval with a sentence; the Restroom Icon Matching task. Lab 2 classified 100 sports from ten labels per class three ways.
- **This deck adds:** where the vector comes out in *every* family (ResNet, ViT, CLIP, DINOv2) and the same photos through six models; the probe as a *measurement*, with the papers' protocol, a C sweep and the few-shot curve; zero-shot in depth: the prompts as a weight matrix, the factor 100, prompt ensembling, the checkpoint; retrieval in depth: task shapes, precision and recall at k, the similarity threshold, search at scale, clustering; the IOAI 2025 Pixel Efficiency task; a self-check.
- **How to use it:** every frame stands alone; every number was measured once with published weights or quoted from a paper. Run each code frame on Day 31's Lab 2 notebook (its cached sports embeddings are the input), then answer the check at the end before reading the answers.

Why Go Deeper

Where the lecture's version stops

- Lab 2 gave you *ten* labels per class and CLIP; a task gives you two, or none, and a choice of encoders. On 5,000 sports photos the nearest-neighbour accuracy runs from 73.3% (ResNet-18) to 94.3% (DINOv2 ViT-S/14, the smallest of six): the space is worth more than any classifier on top of it.
- Your probe scores 0.95 on the training embeddings and 0.62 on validation. The loop is fine; the regularisation is not. The papers sweep it on a validation split.
- Day 31's prompt added 1.8 points on eight classes. On 100 sports, seven templates averaged the paper's way are worth 4.3 points for free.
- "Which photos are duplicates", "which copies came from the same original", "find the match, or say there is none": no classes, no labels. A threshold, a metric for ranked lists and an index are the tools.

After this deck you can

read any model's vector correctly, measure a space with kNN and a probe done properly, build a zero-shot classifier with ensembled prompts and read its probabilities, score and scale retrieval, and plan a zero-shot Olympiad solution.

⇒ Learning Outcomes

By the end of this deck, you will be able to:

- **Read** the embedding out of a ResNet, a ViT, CLIP and DINOv2 at the right layer, with each model's own pre-processing, through one `embed` function.
- **Compare** embedding models on your own photos with leave-one-out kNN before any training, and say which space to use for which photos.
- **Measure** a space with a linear probe the way the papers do: cached features, logistic regression, the regularisation swept on a validation split, and the few-shot curve.
- **Build** a CLIP zero-shot classifier as a weight matrix of prompt embeddings, ensemble prompts in the embedding space, read the probabilities behind the factor 100, and choose the checkpoint.
- **Score** a retrieval system with precision and recall at k and average precision, choose a similarity threshold from a histogram, search a large gallery in chunks or with an index, and cluster embeddings.
- **Plan** a zero-shot Olympiad solution: the prompts, the candidates, the budget, the checks.

Embedding Models: **Where the Vector Comes From**

Definition: An Encoder Read at the Right Place

Embedding model

Any pretrained network read *before* its task head. The fixed-length vector at that point is the embedding; where that point is, and its dimension d, depend on the family.

Family	Where the vector comes out	d	How to read it
ResNet (Day 29)	`avgpool`: the mean of the last feature map over its 7×7 positions	512 (18), 2048 (50)	`model.fc = nn.Identity()`
ViT, torchvision	the `[CLS]` token, an extra learned "patch" that attends to all 196 patches, after the last block's LayerNorm	768 (B/16)	`model.heads = nn.Identity()`
CLIP, image side	the `[CLS]` token, then the `Linear` projection into the space the text shares	512 (B), 768 (L)	`get_image_features`
DINOv2	the `[CLS]` token after the final LayerNorm, no projection	384 (S), 768 (B), 1024 (L)	`model(...).pooler_output`

- Day 29's 512 numbers were one case of the rule: cut below the head, read the vector. A ViT has no `avgpool`; its summary is the `[CLS]` token (Week 6 opens the mechanism).
- **CLIP's projection matters.** The paper's own linear probes use the 768 numbers *before* the projection; zero-shot and text retrieval need the 512 *after* it, where the sentences live. Default to the projected vector.

Six Published Models on One Interface

Model	Trained on	d	Weights	Published number	Library
ResNet-18	ImageNet-1k, 1.28 M labelled photos (supervised)	512	11.7 M	ImageNet top-1 69.8%	torchvision
ResNet-50 (V2 recipe)	the same, longer schedule, stronger augmentation	2048	25.6 M	top-1 80.9%	torchvision
ViT-B/16	the same (supervised)	768	86.6 M	top-1 81.1%	torchvision
CLIP ViT-B/32	400 M (photo, caption) pairs, contrastive	512	151 M (both encoders)	zero-shot ImageNet 63.2%	transformers
CLIP ViT-B/16	the same, 16×16 patches: 196 per photo instead of 49	512	150 M	zero-shot 68.6%	transformers
DINOv2 ViT-S/14	142 M curated photos, **no labels, no captions** (self-supervised: two crops of one photo must agree)	384	22 M	linear probe 81.1%, kNN 79.0%	transformers

- Three kinds of training, one interface: a photo in, a unit vector out. Bigger siblings when the budget allows: CLIP ViT-L/14 (428 M weights, zero-shot 75.3%), DINOv2 ViT-B/14 (86 M, probe 84.5%).
- "Published number" is the model's own headline, not a comparison: top-1 with its own head, zero-shot from prompts, or a probe. The next frames measure all six the same way.

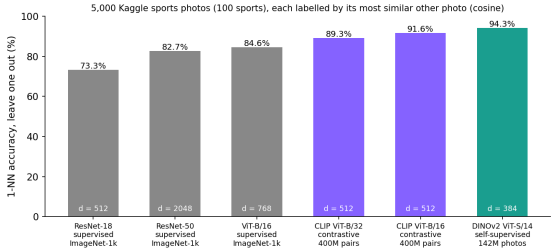
Sources: torchvision model weights documentation; Radford et al. (2021), Table 11; Oquab et al. (2024), Table 4 and the model cards.

In Code: One embed Function for Every Family

```
@torch.no_grad()
def embed(images, family):                          # any model -> (N, d), unit rows
    if family == "torchvision":                     # m = resnet50(weights=w); m.fc = nn.
        Identity()
        e = m(torch.stack([w.transforms()(im) for im in images]))
    elif family == "clip":                          # m = CLIPModel; proc = CLIPProcessor
        e = m.get_image_features(**proc(images=images, return_tensors="pt"))
    elif family == "dinov2":                        # m = AutoModel; proc =
        AutoImageProcessor
        e = m(**proc(images=images, return_tensors="pt")).pooler_output
    e = getattr(e, "pooler_output", e)              # transformers 5 wraps the tensor
    return F.normalize(e.float(), dim=1)
```

- `m.eval()` once after loading, `@torch.no_grad()` on the function (Day 32's second switch): nothing is recorded, nothing kept. For a ViT from torchvision, `m.heads = nn.Identity()` plays the role of `m.fc`.
- The function returns unit rows, so every table after it is `E_a @ E_b.T` (Day 31). Save each model's table with `torch.save`; the models never run twice.
- **One table per model, never mixed:** a ResNet row and a CLIP row are not comparable, even at $d = 512$. Two spaces, two tables.

The Same 5,000 Photos Through Six Models



Source: Kaggle *100 Sports Image Classification* (`gpiosenka/sports-classification`), 50 training photos per sport; six published models applied once, no training; `scripts/day33_embedding_model_figures.py`.

- **Leave-one-out 1-NN** (every photo labelled by its most similar *other* photo) measures a space with no training: ResNet-18 73.3%, ResNet-50 82.7%, ViT-B/16 84.6%, CLIP ViT-B/32 89.3%, CLIP ViT-B/16 91.6%, DINOv2 ViT-S/14 94.3%.
- The order follows the training, not the size: the smallest model (DINOv2, 22 M weights, no labels) wins, the two CLIPs follow, the three ImageNet encoders trail. The first number to compute in a task, for every encoder you can load.

Pre-Processing Belongs to the Model

Model	Resize and crop	Mean	Std
torchvision ResNet, ViT	shorter side 256, centre crop 224	(0.485, 0.456, 0.406)	(0.229, 0.224, 0.225)
CLIP (any checkpoint)	shorter side 224, centre crop 224	(0.481, 0.458, 0.408)	(0.269, 0.261, 0.276)
DINOv2	shorter side 256, centre crop 224	(0.485, 0.456, 0.406)	(0.229, 0.224, 0.225)

- Every published model was trained on tensors made *this* way and its weights expect them; the numbers are `w.transforms()` and the processor's `preprocessor_config.json`. Print them once: `print(w.transforms())`, `print(proc.image_processor)`.
- **The trap:** one `transform` for every model. ImageNet-normalised tensors into CLIP, or CLIP's into DINOv2, still give vectors of the right shape, with no error: the space is simply worse. Load the pre-processing with the weights, as `embed` does, and use the same pipeline for the gallery and the queries.

Sources: torchvision `ImageClassification` transforms; the `preprocessor_config.json` of `openai/clip-vit-base-patch32` and `facebook/dinov2-small`.

Linear Probing **as a Measurement**

What the Probe Measures

Definition

A **linear probe** is the best *linear* classifier on frozen embeddings: scores $z = We + b$, probabilities $\mathrm{softmax}(z)$, $W \in \mathbb{R}^{k \times d}$, $b \in \mathbb{R}^k$, trained on your labels with the encoder untouched. Its validation accuracy measures **how linearly separable your classes are in that space**.

- $k \cdot d + k$ parameters: 100 sports on CLIP's 512 numbers, 51,300 (Lab 2). Seconds to train on cached rows.
- **kNN is the floor:** "the linear probe performance is lower-bounded by the k-NN performance" (Oquab et al., 2024). A low probe says "not linearly separable here"; Day 30's answer is fine-tuning.

With tiny numbers

$d = 2$, $k = 2$, $W = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$, $b = (0, 0)$. A photo with $e = (0.8, 0.2)$: $z = (0.8 - 0.2,\ -0.8 + 0.2) = (0.6, -0.6)$; $\mathrm{softmax}(z) = (1.82, 0.55)/2.37 = (\mathbf{0.77}, 0.23)$: class 1. A photo at $(0.2, 0.8)$ gets $(0.23, 0.77)$: class 2. The boundary is the line $e_1 = e_2$; training moves W until a line separates the labelled photos as well as any line can.

The Protocol the Papers Use

	CLIP (Radford et al., 2021, A.3)	DINOv2 (Oquab et al., 2024, B.3)
features	the penultimate layer, "ignoring any classification layer"; for ViTs the 768 before the projection	the `[CLS]` token of the last block(s), optionally with the mean of the patch tokens
classifier	logistic regression, scikit-learn's L-BFGS, at most 1,000 iterations	one `Linear`, SGD, 12,500 iterations
what is swept	the L2 strength λ from 10^{-6} to 10^{6}, 96 log-spaced steps, binary search to 8 per decade	the learning rate, how many last blocks are read, patch tokens or not
where it is swept	the dataset's validation split, then retrained on train + validation	the validation split
reported	the test metric of the best λ	top-1 on ImageNet's validation set

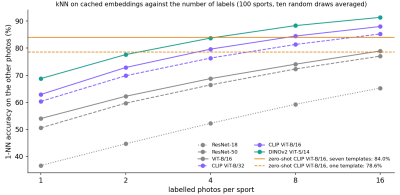
- Two papers, one shape: **frozen features, a linear classifier, the regularisation or rate chosen on validation, the number read on held-out data**. Anything less is not a measurement: a probe at one arbitrary λ can be 10 points off.
- The sweep is cheap because the features are cached: 96 fits of a $512 \times k$ matrix cost seconds. In a contest, five values of C already remove most of the error.

In Code: scikit-learn on Cached Embeddings, C Swept on Validation

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(E_train)                    # mean 0, std 1 per dimension,
    train only
Xtr, Xva = scaler.transform(E_train), scaler.transform(E_val)
best = (0, None)
for C in [0.01, 0.1, 1, 10, 100]:                         # C = 1 / lambda: larger C,
    weaker penalty
    clf = LogisticRegression(C=C, max_iter=1000).fit(Xtr, y_train)
    acc = clf.score(Xva, y_val)
    print(C, round(acc, 3)); best = max(best, (acc, C))
```

- `E_train` is `E.numpy()` from Day 31's cache; `LogisticRegression` *is* the probe (W, b, softmax, cross-entropy) with L-BFGS as the loop. Day 31's torch `Probe` is the same model with Adam; its knob is `weight_decay`.
- **Standardise from the training rows only**; the scaler's mean and std are part of the model.
- Read the printed lines: accuracy rising with C then falling is the shape you want; extend the grid in the direction it still rises. Refit the best C on train + validation (the paper), or keep it.

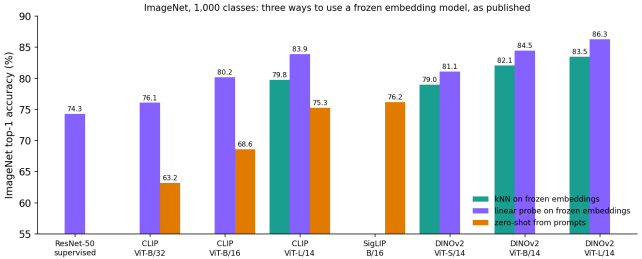
The Few-Shot Curve: Labels Against Accuracy



Source: the same 5,000 Kaggle sports photos; for each count, ten random draws of that many labelled photos per sport, the rest classified by their nearest labelled photo; no training; `scripts/day33_embedding_model_figures.py`.

- **Measured:** one label per sport gives 62.9% with CLIP ViT-B/16 and 68.7% with DINOv2; 16 labels 88.0% and 91.4%. Zero-shot CLIP ViT-B/16 with one template (78.6%) sits where kNN on its own space reaches with about **four** labels per class; with seven templates (84.0%), about eight.
- **Published** (Radford et al., 2021, Figure 6): over 20 datasets zero-shot CLIP equals a **4-shot** probe on its own features and the best **16-shot** probe on other encoders. Wortsman et al. (2022) average zero-shot and fine-tuned weights half and half: ImageNet kept, 3.5 points gained on shifted photos.

Published: kNN, Probe and Zero-Shot on ImageNet



Sources: Radford et al. (2021), Table 10 (linear probes, ImageNet column) and Table 11 (zero-shot); Oquab et al. (2024), Table 4 (kNN and linear, including their measurement of CLIP ViT-L/14); Zhai et al. (2023), Table 3 (SigLIP B/16 zero-shot); `scripts/day33_embedding_model_figures.py`.

- **Labels are worth about ten points over prompts** on one space (CLIP ViT-L/14: zero-shot 75.3, probe 83.9); **the probe two to five over kNN** (DINOv2 ViT-S/14: 79.0 against 81.1). A probe on DINOv2's 22 M frozen weights beats a supervised ResNet-50 with its own head (81.1 against 76.1).
- Read across models with care: CLIP's protocol gives 83.9 for ViT-L/14, DINOv2's 84.3. Numbers from one protocol compare; from two, only large gaps decide.

Pitfalls When Probing

Pitfall	What you see	Fix
no regularisation sweep	train 0.95, validation 0.62	the C grid of the code frame; the shape rises then falls
features not standardised	L-BFGS warns it did not converge; C behaves oddly	`StandardScaler` fit on train, or unit rows for cosine spaces
split after embedding by eye	validation looks great, the test split disagrees	split *paths* by class before embedding (Day 10's rule), embed the splits separately
no augmentation possible	a fine-tuned model with flips beats the probe	embed a few augmented views per photo and add the rows; or accept it: the probe is the floor
test pre-processing differs	accuracy drops on the submission	one `embed` function, one pipeline, for every split
cache in float16	cosines differ in the third decimal	fine for search; keep float32 for the probe's inputs

- The first two account for most "the probe is bad on this space" verdicts. Compare the probe with kNN on the same split: if kNN is higher, the probe is the problem, not the space.
- Keep the three numbers together in your notebook: **kNN, probe, zero-shot** on one validation split. That table decides whether to fine-tune (Day 30) before a minute of GPU is spent.

CLIP Zero-Shot **in Depth**

The Prompts Are the Weight Matrix, and 100 Is Why It Looks Sure

Stack the k unit text embeddings into $W \in \mathbb{R}^{k \times d}$; a unit image embedding u gets

$$z = s\,Wu, \qquad p = \mathrm{softmax}(z), \qquad s = 100, \qquad \text{so } z_j = s\cos(u, v_j)$$

- The probe's $We + b$ with $b = 0$ and W written by the text encoder; $s = 1/\tau$ is a scale learned during CLIP's training (published: 100).
- **Why 100:** a photo's cosines against prompts sit in a narrow band (0.15 to 0.35), so a gap of 0.04 is a real decision; times 100 it is 4 logits, a 50-to-1 preference. The *order* never depends on s; only p does.

With numbers

Three prompts, cosines $(0.31, 0.27, 0.25)$. At $s = 100$: $z = (31, 27, 25)$, $p_1 = 1/(1 + e^{-4} + e^{-6}) = 1/1.020 = \mathbf{0.98}$, $p_2 = 0.018$, $p_3 = 0.002$. At $s = 1$: $p = (0.35, 0.33, 0.32)$, the same winner, no confidence. Cosines $(0.31, 0.30, 0.25)$ at $s = 100$: $p_1 = 1/(1 + e^{-1} + e^{-6}) = \mathbf{0.73}$: a gap of 0.01 is a biased coin.

Reading p: 0.98 means "cosine gap 0.04", not "right 98% of the time". Trust the order and the gap; calibrate on validation if a task scores p.

Prompt Ensembling: Average in the Embedding Space

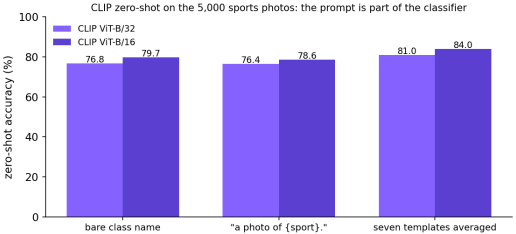
Mechanism

Write m templates ("a photo of a {}.", "a close-up photo of a {}.", ...). For class j, embed the m sentences, **average the unit vectors, normalise again**: one row v_j per class. The classifier is still a $k \times d$ matrix, computed once; prediction costs nothing extra.

```
T = torch.stack([text_embeddings(
        [t.format(c) for c in CLASSES])    #
             (k, d)
     for t in TEMPLATES])                  #
           (m, k, d)
W = F.normalize(T.mean(0), dim=1)          #
    (k, d)
probs = (100 * U @ W.T).softmax(dim=1)
```

- **Why the mean works:** each template adds its own noise ("a photo" pulls towards photographs, "a drawing" towards sketches); the average keeps what the class name contributes and cancels the rest. The paper averages "over the embedding space instead of probability space" so one matrix is cached.
- **Published:** "A photo of a {label}." alone +1.3 points on ImageNet over the bare name; 80 templates averaged +3.5 more; a task hint ("a type of pet") more on fine-grained sets.
- Templates describe *how your photos look* ("a sports photo of {}"), not what the class is: the name says that.

Measured: Three Prompt Choices on 100 Sports



Source: the 5,000 Kaggle sports photos, CLIP ViT-B/32 and ViT-B/16 applied once; the seven templates are listed in `scripts/day33_embedding_model_figures.py`. No training.

- ViT-B/32: bare names 76.8%, one template 76.4%, seven averaged 81.0%. ViT-B/16: 79.7%, 78.6%, 84.0%. The single template is *not* better than the bare name here ("archery" is already a photo-like word); the average of seven adds 4.3 points on both checkpoints, about what the step from B/32 to B/16 buys.
- One template can lose; the ensemble did not, here or in the paper ("reliably improve performance"). Default: a handful of templates, averaged, checked on validation against the bare names, before any label is used.

Choosing the Checkpoint

Checkpoint	Weights	Patches	ImageNet zero-shot	Hugging Face id, under `openai/`
CLIP ViT-B/32	151 M	49	63.2%	`clip-vit-base-patch32`
CLIP ViT-B/16	150 M	196	68.6%	`clip-vit-base-patch16`
CLIP ViT-L/14	428 M	256	75.3%	`clip-vit-large-patch14`
CLIP ViT-L/14 at 336 px	428 M	576	76.2%	`clip-vit-large-patch14-336`
SigLIP B/16	203 M	196	76.2%	`google/siglip-base-patch16-224`

- **Patches decide the cost.** A ViT's work grows with the number of patches: B/16 sees four times as many as B/32 at the same weight count and runs about four times slower; L/14 at 336 px costs about twelve times B/32. Zero-shot accuracy follows the same order.
- **SigLIP** (Zhai et al., 2023) is CLIP with a sigmoid loss on each pair instead of a softmax over the batch: the same two encoders and `get_image_features`, better numbers at the same size; its scores carry a learned scale *and* bias and go through a sigmoid, not a softmax.
- In a timed task: embed the gallery once with the biggest checkpoint the budget allows (Day 32's arithmetic) and cache; prompts are free to change afterwards.

Sources: Radford et al. (2021), Table 11; Zhai et al. (2023), Table 3; the model cards (weight counts).

Retrieval **in Depth**

Retrieval in Disguise: Five Task Shapes

Task shape	Query and gallery	Answer	The extra step
search	one photo or sentence against N photos	the top k rows of S	none; score with precision or recall at k
matching (Day 31's icons)	M queries against N candidates	one gallery item per query	use the structure: forbid the impossible, assign each candidate once
duplicates	every photo against every other	pairs with $S_{ij} > t$	choose t from the histogram
"no match" allowed	queries against a gallery	the best item, or *none*	the same threshold on the best score
clustering	no query: all photos	a group id per photo	k-means on the unit rows; the groups are the answer

- All five start with the same two lines (embed, normalise) and one table S; they differ in what is read off it. Recognising the shape in a statement is most of the solution.
- Labels, when a task gives them, are for *scoring* these systems and for choosing t or k, not for training: the encoder is frozen throughout. With many labels per class, go back to the probe.

Scoring Retrieval With Numbers

Definitions for one query with R relevant gallery items

Precision at k: relevant among the top k of the ranked gallery, over k. **Recall at** k: relevant among the top k, over R. **Average precision (AP)**: the mean, over the relevant items, of the precision at the rank where each appears. **mAP**: AP averaged over the queries. With $R = 1$, recall at k is "the right item is in the top k".

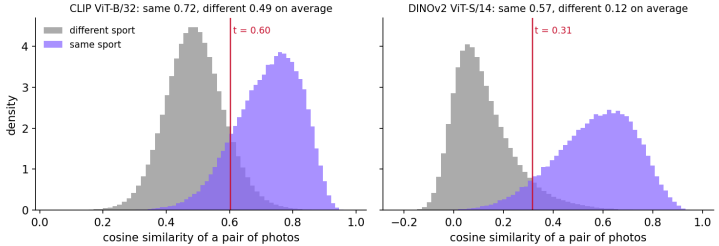
Precision asks "is the top of the list clean", recall "did we find them all", AP both at once. Published: zero-shot CLIP, Flickr30k, text to image: R@1 68.7, R@5 90.6, R@10 95.2.

With numbers

A gallery of 10 photos, $R = 3$ show the query's sport. Ranked by cosine, they land at ranks **1**, **3** and **7**.

- P@5 $= 2/5 = 0.40$; R@5 $= 2/3 = 0.67$; R@10 $= 1$.
- Precision at the relevant ranks: $1/1 = 1.00$, $2/3 = 0.67$, $3/7 = 0.43$; AP $= (1.00 + 0.67 + 0.43)/3 = \mathbf{0.70}$.
- At ranks 1, 2, 3: AP $= 1$; at 8, 9, 10: AP $= 0.22$. Same R@10, very different lists.

The Similarity Histogram and the Threshold



Source: all pairs among the 5,000 Kaggle sports photos (12.5 million), CLIP ViT-B/32 and DINOv2 ViT-S/14 embeddings; the line is the cosine t that best separates same-sport from different-sport pairs; `scripts/day33_embedding_model_figures.py`.

- Two hills, one per kind of pair: same sport 0.72 on average against 0.49 for CLIP ViT-B/32; 0.57 against 0.12 for DINOv2. The hills overlap, so every t trades two mistakes: $t = 0.60$ keeps 87% of the same-sport pairs and rejects 89% of the others (CLIP).
- The threshold is **a property of the model**: 0.6 means "same sport" in one space and "nearly identical" in another. Choose it on labelled pairs from the validation split, for the model you ship; for duplicates, read the pairs just above t before trusting it.

Exact Search at Scale: The Arithmetic, Then the Index

One query against N unit rows of d numbers costs $N \cdot d$ multiply-adds; the gallery takes $4Nd$ bytes in float32. $N = 10^4$, $d = 512$: 20 MB, 5×10^6 products; $N = 10^6$: 2 GB, 5×10^8. Day 31's $N \times N$ table would be 4 TB at $N = 10^6$: keep the gallery, never the table.

```
G = G.half().to(device)                  #
    gallery, unit rows
hits = []
for i in range(0, len(Q), 1024):         #
    queries in chunks
    S = Q[i:i + 1024].half().to(device) @ G.
        T
    hits.append(S.topk(10, dim=1).indices.
        cpu())
idx = torch.cat(hits)                    # (
    N_query, 10)
```

- **Chunk the queries**, keep only each chunk's `topk`. A GPU does 10^{13} products a second: a million-row gallery answers a query in under a millisecond; a contest gallery of 10^4 rows in nothing.
- **FAISS** (Johnson et al., 2019) is this loop as a library: `IndexFlatIP` is exact (a matrix product and a k-selection); `IndexIVFFlat` searches a few cells of a partitioned space, trading a little recall for speed; compressed indexes fit a billion vectors in memory.
- Approximate indexes miss the true neighbour sometimes: check the recall on validation first. Below 10^6 rows the chunked loop is exact and fast enough.

Clustering Embeddings: When Nobody Named the Classes

Definition

Clustering (Day 08's k-means) on unit embeddings: find K centres so that each photo sits near one; the group id is the output. No labels in, group ids out. On unit rows Euclidean and cosine order agree, so k-means fits.

```
from sklearn.cluster import KMeans
from sklearn.metrics import
    adjusted_rand_score
E = embed(images, "dinov2").numpy()   # unit
     rows
km = KMeans(n_clusters=32, n_init=10,
    random_state=0)
groups = km.fit_predict(E)            # (N,)
     ids 0..31
print(adjusted_rand_score(y_val, groups[val
    ]))
```

- **Scoring without names:** the **adjusted Rand index** (ARI) compares two groupings pair by pair: 1 when every pair is together or apart in both, 0 for a random grouping. Ids need not match.
- **NEOAI 2025, Cluster Pictures:** thousands of heavily augmented copies of **32 originals**, a fixed embedding network, 32 groups, scored by ARI: the code on the left with the given network.
- **Which space:** the one whose same-origin pairs score highest on the histogram (DINOv2 was trained to make two crops of one photo agree). K unknown: try several, read the silhouette (Day 08).

Sources: NEOAI 2025, task 4, *Cluster Pictures* (Kaggle); scikit-learn `KMeans` and `adjusted_rand_score`.

Zero-Shot: **Olympiad Practice**

Practice Task: Keep 6.25% of the Pixels

Task

Camera-trap photos (224×224) must cross a slow link. You get **700 training photos with the species name** and a **CLIP ViT-L/14** that names species zero-shot. For each of **698 unnamed test photos** return *one rectangle* of at most **6.25%** of the pixels (a 56×56 square). Everything outside is painted black, and the masked photo goes through the same CLIP with the species names *plus a class "other"*. Score: zero-shot accuracy on the masked test photos, **within 8 minutes** for all 698; baseline (centre square) 0.19, best 0.83.

- **Question 1:** what is the classifier here, what may you change, and what do you compute exactly once?
- **Question 2:** the test photos carry no name. How do you choose a rectangle for a photo whose answer you do not know, and what is "other" for?
- **Question 3:** a 56×56 window at stride 28 gives 49 candidates per photo: 34,000 masked photos through ViT-L/14. Does that fit 8 minutes, and what if not?

Source: IOAI 2025 (Beijing), Individual Contest, Task 6, "Pixel Efficiency" (`github.com/IOAI-official/IOAI-2025`); every number from the statement.

Solution Idea and Code

```
W = text_embeddings(SPECIES + ["other"])      # (k+1, 768): the grader's prompts
def best_window(img, side=56, stride=28):
    u = embed([img], "clip")                  # (1, 768): the whole photo
    label = (u @ W.T).argmax()                # zero-shot: the answer to keep
    grid = range(0, 224 - side + 1, stride)   # 0, 28, ..., 168
    tops = [(t, l) for t in grid for l in grid]          # 49 windows
    masked = [black_outside(img, t, l, side) for t, l in tops]
    P = (100 * embed(masked, "clip") @ W.T).softmax(1)   # (49, k+1)
    t, l = tops[P[:, label].argmax()]         # keeps the label surest
    return ((t, l), (t + side, l + side))
```

1. **The classifier is fixed:** CLIP's image side and the grader's prompts (bare names plus "other"); you choose only the pixels. W once, the whole photo once; keep the "other" row: a window that scores "other" first earns nothing.
2. **The label you do not have:** zero-shot on the *whole* photo is right most of the time; rank windows by p_{label} of the masked photo so the window keeps *that* answer. Measure on the 700 named photos.
3. **The budget (Day 32):** time one photo (49 crops, one float16 batch), times 698, keep a margin; over the limit, stride 56. The committee's best scored every rectangle from *one* pass with extra tokens, then verified its top three with real masked passes.

Embedding Models: **Check Yourself**

Check Yourself

Answer before turning the page.

1. **Shapes.** 64 photos go through torchvision's ViT-B/16 with `heads = nn.Identity()`, through DINOv2 ViT-S/14 (`pooler_output`) and through CLIP ViT-B/32 (`get_image_features`). What are the three shapes, and may you put the three tables into one similarity matrix?
2. **The temperature.** Two prompts; a photo's cosines are 0.28 and 0.26. What probability does CLIP report for the first prompt with the published scale $s = 100$, and what would it be with $s = 1$? Does the winner change?
3. **The probe.** A probe at $C = 100$ scores 0.95 on the training embeddings and 0.62 on validation; kNN on the same split scores 0.74. What is wrong, which two changes do you make, and what do you expect the probe to score afterwards relative to kNN?
4. **Retrieval.** A query has $R = 3$ relevant photos in a gallery of 10; ranked by cosine they land at ranks 2, 4 and 5. Compute P@5, R@5, R@1 ("is a relevant item first") and the average precision.

Answers

1. $(64, 768)$, $(64, 384)$ and $(64, 512)$: the `[CLS]` token of ViT-B, DINOv2's `[CLS]` after its LayerNorm, CLIP's projected vector. No: three models, three spaces. Rows from different models are not comparable even when the widths agree; one S per model.
2. $z = (28, 26)$, $p_1 = 1/(1 + e^{-2}) = 1/1.135 = \mathbf{0.88}$. With $s = 1$: $z = (0.28, 0.26)$, $p_1 = 1/(1 + e^{-0.02}) = \mathbf{0.505}$. The winner is the same; only the reported confidence changes, which is why a p of 0.88 means "cosine gap 0.02" and not "right 88% of the time".
3. The probe overfits: $C = 100$ is almost no regularisation ($\lambda = 0.01$), and a probe below kNN on the same split is a probe problem, not a space problem. Standardise with a scaler fitted on the training rows, then sweep C downwards $(10, 1, 0.1, 0.01)$ and read the validation accuracy; keep the C where it peaks. Expect the probe at or a few points above kNN's 0.74; if it stays below, check that the split and the pre-processing are the same for both.
4. Top 5 holds the items at ranks 2, 4, 5: P@5 $= 3/5 = 0.60$; R@5 $= 3/3 = 1.0$; R@1 $= 0$ (rank 1 is not relevant). AP $= (1/2 + 2/4 + 3/5)/3 = (0.50 + 0.50 + 0.60)/3 = \mathbf{0.53}$: perfect recall at 5, but the top of the list is not clean.

Embedding Models: **Limitations & What Comes Next**

Limitations

- **One dataset is one data point.** The sports photos put DINOv2 first and ResNet-18 last; satellite tiles, X-rays or drawings can reorder the six. The measurement is cheap (leave-one-out kNN, no training): repeat it on *your* photos rather than trusting this order.
- **The published table mixes protocols.** CLIP's probe of ViT-L/14 says 83.9, DINOv2's 84.3; DINOv2's kNN uses a weighted vote, the deck's a single neighbour. Within one protocol numbers compare; across protocols only large gaps mean anything.
- **Zero-shot probabilities are not calibrated**, and SigLIP's sigmoid scores are not probabilities over classes at all. A task that scores a probability needs a calibration step on validation, which this deck only names.
- **Frozen means frozen.** Adapting CLIP itself (fine-tuning its image side, small trainable layers, averaging weights as WiSE-FT does) is named, not covered; the deck shows exact search only.
- **Bias and ambiguity travel with the prompts** (Day 31): "boxer", "crane", "bat". Averaging templates does not fix a wrong word; a context phrase does.

The common mistake

Judging a space by a probe trained once, at one C, on unstandardised features, then fine-tuning for an hour to beat a number that a two-line kNN would have beaten in a second. Measure first: kNN, the swept probe, zero-shot, on one validation split.

What Comes Next

- **Day 34, Olympiad practice:** small-data fine-tuning habits: what wins points and what loses them in a Friday-style task, with the sources; where today's cached-embedding tools sit in that plan.
- **Friday:** the Week 5 competition.
- **Week 6:** the same ideas for text: a sentence as an embedding, cosine between sentences, semantic search with the same `topk`, TF-IDF against learned embeddings, and the transformer that CLIP's text side and DINOv2's image side are made of.
- **Practice now:** open Day 31's Lab 2, add DINOv2 ViT-S/14 and CLIP ViT-B/16 through the `embed` function, print the leave-one-out kNN of each space, run the C sweep on the probe, average seven templates, and draw the histogram of same-sport against different-sport cosines for the model that won.

Embedding Models: **Summary**

Summary

- **Embedding models:** cut below the head and read the vector (`avgpool`, the `[CLS]` token, CLIP's projection), with each model's own pre-processing; one `embed` function, one table per model. On 5,000 sports photos: 73.3% (ResNet-18) to 94.3% (DINOv2) by kNN.
- **Linear probing** measures linear separability in a space: cached features, logistic regression, C swept on validation, standardised inputs; kNN is its floor; labels beat prompts by about ten points (CLIP ViT-L/14: 75.3 zero-shot, 83.9 probed).
- **Few labels:** one per sport gave 62.9% with CLIP ViT-B/16 by kNN; zero-shot (78.6%) equals kNN with about four labels (the paper's 4-shot result); the probe overtakes zero-shot from 16.
- **Zero-shot in depth:** $z = 100\,Wu$ with W the stacked prompt embeddings; 100 turns a cosine gap of 0.04 into $p = 0.98$: read the order and the gap. Seven templates averaged: $+4.3$ points. Patches, not weights, set the cost.
- **Retrieval in depth:** five task shapes on one table S; precision and recall at k, AP, mAP; a threshold from the histogram of same against different pairs, per model; chunked exact search or an index; k-means on unit rows, scored by ARI.
- **In a task:** embed once with the best space the budget allows, then kNN, the swept probe and ensembled zero-shot on one validation split; in Pixel Efficiency the prompts are the grader's, and a window must keep the whole-photo answer.

Embedding Models: **References & Resources**

References & Resources

[1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *ICML*. Section 3.1.4 (prompt engineering and ensembling: $+1.3$, $+3.5$, 80 templates, averaged in the embedding space), Figure 6 (zero-shot against 4-shot and 16-shot probes), Appendix A.3 (the linear probe protocol: penultimate features, L-BFGS, λ swept from 10^{-6} to 10^{6} on validation), Table 10 (linear probes, ImageNet column), Table 11 (zero-shot by checkpoint), Table 13 (Flickr30k retrieval). https://arxiv.org/abs/2103.00020

[2] Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A. and Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *TMLR*. Table 1 ("the linear probe performance is lower-bounded by the k-NN performance"), Table 4 (kNN and linear probes on ImageNet, including CLIP ViT-L/14), Appendix B.3 (the linear probing grid), Table 17 (embedding dimensions 384 / 768 / 1024 / 1536). https://arxiv.org/abs/2304.07193

[3] Zhai, X., Mustafa, B., Kolesnikov, A. and Beyer, L. (2023). Sigmoid loss for language image pre-training. *ICCV*. Table 3 (SigLIP B/16 zero-shot ImageNet 76.2%; CLIP B/16 68.3% in the same table). https://arxiv.org/abs/2303.15343

[4] Wortsman, M., Ilharco, G., Kim, J. W., Li, M., Kornblith, S., Roelofs, R., Gontijo Lopes, R., Hajishirzi, H., Farhadi, A., Namkoong, H. and Schmidt, L. (2022). Robust fine-tuning of zero-shot models. *CVPR*. Table 1 (WiSE-FT, $\alpha = 0.5$: 86.8 on ImageNet, 76.9 on the shifts, against 76.2 / 73.3 zero-shot and 86.2 / 68.6 fine-tuned). https://arxiv.org/abs/2109.01903

References & Resources (cont.)

[5] Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Trans. Big Data*. Exact search as a matrix product plus k-selection (their equation 2), a k-NN graph of 95 million images in 35 minutes, one billion vectors in under 12 hours on four GPUs. FAISS: https://github.com/facebookresearch/faiss

[6] Dosovitskiy, A. et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*. The `[CLS]` token read as the ViT's embedding (opened in Week 6).

[7] torchvision models documentation (`ResNet18_Weights`, `ResNet50_Weights.IMAGENET1K_V2`, `ViT_B_16_Weights`: accuracies, parameter counts, `transforms()`). https://pytorch.org/vision/stable/models.html Hugging Face model cards and `preprocessor_config.json` of `openai/clip-vit-base-patch32`, `openai/clip-vit-base-patch16`, `openai/clip-vit-large-patch14`, `google/siglip-base-patch16-224`, `facebook/dinov2-small`. scikit-learn: `LogisticRegression` (L-BFGS, `C`), `StandardScaler`, `KMeans`, `adjusted_rand_score`.

[8] IOAI 2025 (Beijing), Individual Contest, Task 6, "Pixel Efficiency" (the practice task: CLIP ViT-L/14, 700 / 698 photos, one rectangle of at most 6.25%, the "other" class, 8 minutes, baseline 0.19, best 0.83) and its reference solution. https://github.com/IOAI-official/IOAI-2025 NEOAI 2025, Task 4, "Cluster Pictures" (32 originals, a fixed embedding network, adjusted Rand index). https://www.kaggle.com/competitions/neoai-2025-cluster-pictures

[9] Kaggle dataset *100 Sports Image Classification* (`gpiosenka/sports-classification`; 50 training photos per sport, 5,000 in all) through `kagglehub`. Figures without another source: `slides-builder/scripts/day33_embedding_model_figures.py`: six published models applied once to those photos (leave-one-out kNN, few-shot kNN over ten random draws, zero-shot with three prompt choices, the pair histograms), no training; the ImageNet chart redraws the published tables above.